

Universidade Federal do Rio Grande do Norte
Departamento de Informática e Matemática Aplicada
DIM0548 – Engenharia de Linguagens

DOCUMENTAÇÃO

KOJITO

EXPEDITO HEBERT FIRMINO DA ROCHA
FRANCISCO GABRIEL COSTA BESSA
JOSE CARLOS DA SILVA NASCIMENTO
PEDRO VINÍCIUS BARBOSA PEREIRA
SABRINA DA SILVA BARBOSA VENCESLAU

Natal - RN
2026

SUMÁRIO

1. APRESENTAÇÃO DA LINGUAGEM PROPOSTA	3
2. DESIGN DA IMPLEMENTAÇÃO	4
2.1 Transformação do código-fonte em unidades léxicas	4
2.2 Representação de símbolos, tabela de símbolos e funções associadas	7
2.3 Tratamento de estruturas condicionais e de repetição	7
2.4 Tratamento de subprogramas	9
2.5 Verificações realizadas (tipos, faixas, declaração em duplicidade, etc)	10
3. INSTRUÇÕES DE USO DO COMPILADOR	10

1. APRESENTAÇÃO DA LINGUAGEM PROPOSTA

O presente trabalho tem como objetivo apresentar a linguagem de programação Kojito, que está sendo desenvolvida como parte dos requisitos para concluir a disciplina de Engenharia de Linguagem do Departamento de Informática e Matemática Aplicada/DIMAP na Universidade Federal do Rio Grande do Norte/UFRN.

Para tanto, a Kojito consiste em ser uma linguagem de programação com paradigma imperativo que se fundamenta na arquitetura de von Neumann e adota um estilo de programação com uso de variáveis, instruções de atribuição e na forma iterativa de repetição. De modo geral, caracteriza-se pela forma de comandos detalhados para a realização de uma determinada tarefa.

Também é importante salientar que o domínio de aplicação da nossa linguagem é de uma Linguagem educacional, com foco no ensino de aspectos relacionados à segurança de memória, bem como aspectos que vêm do funcional. Nossa ideia se aplica principalmente a alunos que já tiveram algum contato com programação, especialmente quem já teve algum contato com programação no nível do que é visto, por exemplo, na disciplina de Linguagem de Programação 1, ofertada pelo Instituto Metr pole Digital/IMD no curso do Bacharelado em Tecnologia da Informa  o BTI. Dessa forma, o objetivo   dar um suporte no aprendizado sobre coisas b sicas, por m importantes para um bom uso dos recursos computacionais, ressaltando os trade offs relacionados ao uso de mem ria e os custos ao utilizar recursos de alto n vel.

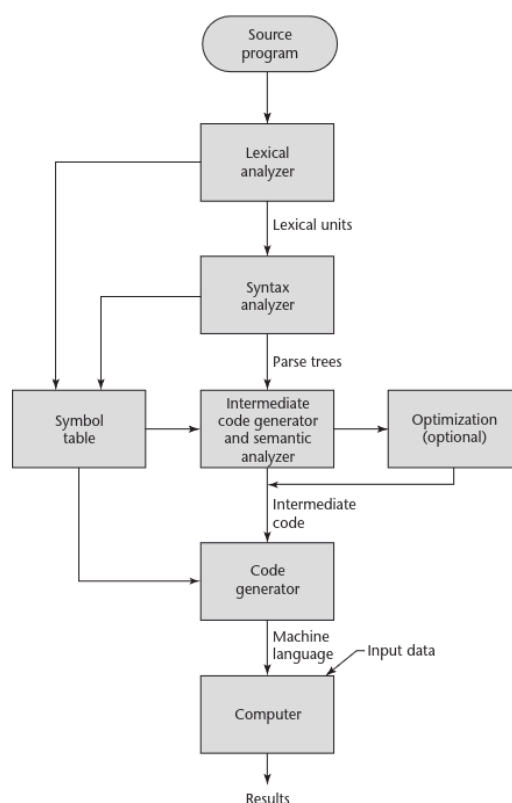
Salienta-se ainda que a Kojito est  no  mbito de software b sico educacional e, baseando-se no livro *concepts of programming Languages* edi  o 11 de Robert W. Sebesta, os crit rios que t m maior relev ncia para a nossa linguagem s o, principalmente, a legibilidade e a confiabilidade. Destacando que algumas caracter sticas presentes em capacidade de escrita, como o tipo de dados, ortogonalidade e suporte   abstra  es, por exemplo, t m um papel fundamental para a constru  o da nossa linguagem, pois essas caracter sticas somadas a checagem de tipo, tratamento de exce  o e restricted aliasing, fornecem uma estrutura robusta para ela. Todavia, apesar de n o ser uma linguagem com foco em alta performance, algumas caracter sticas relacionadas ao custo se fazem necess rias para se ter um equil brio entre o uso de recursos dispon veis (baixo n vel) e a contribui  o ao aprendizado do usu rio (alto n vel). Entretanto, caso necess rio, essas caracter sticas de custo s o as primeiras a serem penalizadas em prol de uma maior confiabilidade e/ou legibilidade.

Por fim, cabe destacar que para desenvolver a linguagem de programa  o Kojito, n o ser  necess rio criar uma nova camada de abstra  o, sendo poss vel usufruir das caracter sticas do paradigma imperativo e ainda deix -la num n vel intermedi rio entre alto n vel e baixo n vel. Portanto, nossa ideia consiste em uma linguagem mais segura, tendo foco em legibilidade e confiabilidade, sacrificando aspectos, principalmente, relacionados ao custo se preciso for.

2 DESIGN DA IMPLEMENTAÇÃO

Para a concretização de nossa ideia seguimos as instruções norteadoras dadas pelo professor Dr. Umberto Souza da Costa, bem como baseamo-nos no fluxo presente no livro do Sebesta - Concepts of programming Languages edição 11, que apresenta o processo de compilação. Dessa forma, iniciamos a nossa linguagem definindo o que seria um programa nela em que, a partir disso, construímos os nossos tokens para gerar a nossa tabela de símbolos, para então gerar as estruturas sintáticas da mesma e, por fim, poder realizar as relações de sentido entre as estruturas sintáticas, bem como as relações de tipos, conforme imagem abaixo:

Figura 1 – Processo de compilação



Fonte: Robert W. Sebesta (2015).

Com isso, entraremos em detalhes a seguir, de modo que traremos exemplos de como ocorre em nossa linguagem, apresentando quais foram as estratégias que utilizamos para a realização desses passos.

2.1 Transformação do código-fonte em unidades léxicas

Para que um código em linguagem imperativa seja lido e executado é importante lembrar que ele passa por 3 grandes etapas que são popularmente conhecidas por: Analisador léxico; Analisador Sintático e Analisador Semântico, além de passar por outros processos menores como podemos ver na figura 1.

Isso posto, de acordo com Andrew W. Appel (1998), o analisador léxico recebe um fluxo de caracteres e produz um fluxo de nomes, palavras-chaves e sinais de pontuação (lexemas), descartando

espaços em branco e comentários entre os tokens. Porém, cabe destacar que a depender do design da linguagem, pode acontecer de espaços em branco serem na verdade tokens, não podendo descartá-los. Complementar a isso, de acordo com Sebesta (2015), um analisador léxico é essencialmente um “pattern matcher”, ou seja, é algo que realiza casamento de padrões.

Dessa forma, para uma dada entrada são realizadas comparações com os símbolos existentes na linguagem, de modo que a partir dessas comparações os tokens serão associados ao conjunto de caracteres detectado. Dito isso, as principais tarefas de um analisador léxico são:

- Ler (escanear) todos os caracteres de um dado arquivo (fluxo de entrada);
- Identificar lexemas (agrupamento de símbolos conhecidos) a partir dos caracteres lidos;
- Gerar tokens (unidade gramatical de uma linguagem) a partir dos lexemas;
- Ignorar o que é considerado desnecessário;
- Gerar a tabela de símbolos para o fluxo de entrada dado;
- Detectar caracteres não reconhecidos e comunicar erro de símbolo que não faz parte do conjunto de símbolos aceitos pela linguagem.

Assim, realizadas essas tarefas e não havendo nenhum erro lexical, será possível avançar para a etapa de análise sintática, da qual depende essencialmente do que o analisador léxico produz.

Na nossa linguagem, utilizamos regras sintáticas para as quais se torna possível o diálogo com a tabela de símbolos. Cada token possui uma estrutura que o define, sendo possível identificar como devem ser utilizados na linguagem.

Inicialmente, definimos o que é um programa em nossa linguagem. Para isso, utilizamos a regra sintática "Program" como o ponto de partida do compilador. Nesse sentido, é nesta regra inicial que são colocadas as bibliotecas de C que utilizamos (stdio.h e stdbool.h), dado que nossa linguagem está sendo traduzida para C simplificado. Nessa estrutura, o "\$1" refere-se ao bloco geral de declarações (Decls) e entrega o seu atributo "code" (um ponteiro de char contendo todo o código traduzido do programa) para ser escrito diretamente no arquivo de saída através da função fprintf.

```
Program:
  Decls {
    fprintf(yyout, "#include<stdio.h>\n#include<stdbool.h>\n%s", $1-
>code);
  }
;

Decls: SubProgram Decls{
    char* temp[] = {$1->code, "\n", $2->code};
    $$ = CreateRecord(cat(temp,3));
}
| Assignment Decls{
    char* temp[] = {$1->code, "\n", $2->code};
    $$ = CreateRecord(cat(temp,3));
```

```

    }
    | StructDecl Decl{
        char* temp[]={$1->code, "\n", $2->code};
        $$ = CreateRecord(cat(temp,3));
    }
    | EnumDecl Decl{
        char* temp[] = {$1->code, "\n", $2->code};
        $$ = CreateRecord(cat(temp,3));
    }
    | Main {
        $$=CreateRecord($1->code);
    }
    ;

```

Com isso, é perceptível que, para nossa linguagem, um programa é definido por declarações e essas declarações podem ser dadas a partir de subprogramas, atribuições (*assignments*), *structs*, *enums* e pela própria *main*.

Além disso, ao olhar para a regra de Decl se define por meio de uma estrutura recursiva à direita. Para cada uma das opções (como "SubProgram Decl" ou "Assignment Decl"), as informações do elemento atual da declaração ficam guardadas em \$1, enquanto o restante das declarações seguintes fica em \$2. Os códigos gerados por essas duas partes são concatenados e mandados para o símbolo de retorno \$\$ (que representa o próprio Decl).

Dessa forma, o \$2 funciona como o laço recursivo que varre as demais instruções do código-fonte, enquanto o \$1 alterna conforme o tipo de comando que o programador está declarando naquela linha. Salienta-se ainda que, para algumas regras, há um maior detalhamento à medida em que se tornam mais complexas, como por exemplo a regra do escopo:

```

Scope: '{' { PushScope(scopeStack, GenerateScope()); } '}' {
    $$ = CreateRecord("{}"); PopScope(scopeStack);
}
| '{' { PushScope(scopeStack, GenerateScope()); } Statements '}' {
    char* temp[] = {"\n\t",$3->code, "\n"};
    $$ = CreateRecord(cat(temp,3));
    $$->returnType=$3->returnType;
    PopScope(scopeStack);
}

```

Nesse sentido, o escopo (*Scope*) tem sua estrutura definida como chaves (*{}*) podendo estar vazio ou conter um bloco de comandos internos (*{ statments }*). Cabe destacar ainda que as ações semânticas referentes à pilha de escopos (*scopeStack*) são utilizadas principalmente para gerenciar o tempo de vida e a visibilidade das variáveis. Ao empilhar um novo escopo com *PushScope* ao abrir as chaves e desempilhá-lo com *PopScope* ao fechá-las, o compilador evita o acesso indevido a informações que pertencem a contextos distintos.

Em geral, as regras definidas no *parser* seguem esse processo padrão de concatenar as informações de forma semelhante. Todavia, elas possuem particularidades características que exigem uma maior atenção, como foi mostrado. Desde que o registro esteja sendo gerado corretamente, o compilador será capaz de avaliar o fluxo de entrada feito na nossa linguagem, realizar o casamento de padrões e identificar erros sintáticos ou semânticos, caso existam.

2.2 Representação de símbolos, tabela de símbolos e funções associadas

Para a tabela de símbolos, foi feita uma tabela hash. Através das funções *create_table*, *hash*, *insert_symbol*, *lookup_symbol* torna-se possível registrar todas as informações necessárias para serem consultadas durante a compilação de um programa na nossa linguagem. Dessa forma, além das regras sintáticas, há também a tradução das estruturas da nossa linguagem para as estruturas de C simplificado, utilizando o GCC para gerar o executável final. Um exemplo prático do uso da tabela hash pode ser visualizado abaixo na regra de acesso a arrays:

```
Array: ID ArrayAccesses {
    type typeArrayAccess = getVarType($1);
    for(int i = 0; i < $2->sizeofArrayAccess; i++){
        type arrayStore = lookup_symbol(typeTable, typeArrayAccess)-
>info->isArray;
        if(arrayStore != NULL){
            typeArrayAccess = arrayStore;
        }
    }
    char temp[] = {$1, $2->code};
    $$ = CreateRecordType(cat(temp,2), typeArrayAccess);
}
;
```

No trecho acima, a função de busca *lookup_symbol* é utilizada na tabela de tipos para resolver as dimensões do array a cada nível de acesso. Nesse sentido, são passados para essa função a tabela e o tipo atual da variável, permitindo atualizar a variável *typeArrayAccess* com o tipo interno do array, que é temporariamente atribuído a *arrayStore*. Ao final do loop, garante-se que o tipo resultante corresponda ao dado que está sendo efetivamente acessado, seja ele um tipo base ou um ponteiro compatível, permitindo a criação correta do registro na árvore sintática.

Para as funções de inserção, criação de tabela e criação do hash, foram distribuídas em arquivos distintos para modularizar o projeto e facilitar a leitura do *parser*. Todavia, existe uma total integração entre esses arquivos, permitindo que o registro e a consulta na tabela de símbolos sejam realizados com sucesso, desde que o fluxo de entrada esteja coerente com as regras estabelecidas para a linguagem Kojito.

2.3 Tratamento de estruturas condicionais e de repetição

Em relação às estruturas condicionais temos o If e as comparações que fazem parte da estrutura ($=$, $!=$, $>$, $<$, $>=$, $<=$). Dessa forma, para melhor entender como funciona o if em nossa linguagem, abaixo há um exemplo de um if simples:

```
DecisionStructures: IF '(' Expression ')' Scope {
    char* counter = ifCount();
    char* tempif[] = {"IF_SCOPE", counter};
    char* tempend[] = {"ENDIF_", counter};
    char* ifScope = cat(tempif, 2);
    char* endIf = cat(tempend, 2);
    char* temp[] = {
        "\nif", "(", $3->code, ")", " goto ", ifScope,
";\n",
        "\ngoto ", endIf, ";\n",
        ifScope, ":\n", $5->code, "\n",
        endIf, ": "
    };
    $$ = CreateRecord(cat(temp, 16));
}
```

Nesse sentido, esse trecho indica que o if tem uma estrutura simples: *if (expression) scope* e, logo adiante, se tem como é realizada a tradução dessa estrutura para o C simplificado.

Assim, a função *ifCount()* gera um contador único para que cada estrutura if no programa tenha seus próprios rótulos (*IF_SCOPE* para o bloco interno e *ENDIF_* para o fim do bloco), evitando conflitos entre múltiplos condicionais.

Para o mapeamento do fluxo, se a condição ($\$3 \rightarrow code$) for verdadeira, o programa faz um desvio (*goto*) para o rótulo do escopo interno (*ifScope*) e executa o código do bloco ($\$5 \rightarrow code$). Caso contrário, se a condição for falsa, o programa ignora o desvio condicional, encontra o comando *goto endIf*; e salta diretamente para o final da estrutura, ignorando o corpo do if.

Por fim, o array temp concatena todas essas instruções (a validação da expressão, as instruções de salto e o código do escopo) em uma única string que substitui a regra sintática na árvore de derivação.

Além disso, para as estruturas de repetição temos o exemplo do *while* que se estrutura da seguinte forma: *while (expression) scope* e sua estrutura em C simplificado é dada conforme regra abaixo:

```
RepeatStructures:
    WHILE '(' Expression ')' Scope {
        char* counter = whileCount();
        char* tempw[] = {"WHILE_", counter};
        char* tempendw[] = {"ENDWHILE_", counter};
        char* tempwhileScop[] = {"WHILESCOPE_", counter};
        char* whileLabel = cat(tempw, 2);
        char* endWhile = cat(tempendw, 2);
        char* whileScope = cat(tempwhileScop, 2);
```



```

        char* temp[] = {
            "\n", whileLabel, ":\nif(", $3->code, ")", "
goto ", whileScope, ";\n",
            "goto ", endWhile, ";\n",
            whileScope, ":\n", $5->code, " ",
            "\ngoto ", whileLabel, ";\n",
            endWhile, ":"
        };
        $$ = CreateRecord(cat(temp, 20));
    }

```

Dessa forma, para o *while* a tradução é feita de modo que a função *whileCount()* gera um identificador numérico único para aquele laço, semelhante ao que foi feito para o *if*. A partir dele, são criados três rótulos específicos usando a função *cat*:

- *whileLabel* (WHILE_X): O ponto de início e reavaliação da condição.
- *whileScope* (WHILESCOPE_X): O início do bloco de código interno do laço.
- *endWhile* (ENDWHILE_X): O ponto de saída do laço.

Assim, o array *temp* organiza a lógica do laço de modo que no *whileLabel* é avaliada a condição (*\$3->code*). Se for verdadeira, dispara um *goto* para o corpo do laço (*whileScope*). Se for falsa, o código segue para a linha seguinte e dispara um *goto* para fora do laço (*endWhile*). Em seguida o código interno do *whileScope* corresponde ao escopo do *while* (*\$5->code*). Assim, ao final do bloco do escopo, o *goto whileLabel*; força o programa a voltar para o início para reavaliar a condição, criando o ciclo de repetição.

Por fim, o array *temp* (com suas 20 strings concatenadas) é processado pela função *CreateRecord*, salvando a estrutura do *while* em C simplificado no nó atual da árvore sintática (\$\$).

2.4 Tratamento de subprogramas

A linguagem suporta três construções distintas para subprogramas, diferenciadas por suas palavras-chave e comportamentos semânticos:

- **FUNCTION**: Declara uma função padrão que recebe parâmetros, possui um tipo de retorno (indicado por *ARROW Type*) e executa um bloco de código (*Scope*). A função *FUNCTION_Decl* gerencia seu registro.
- **PURE FUNCTION**: Uma variação que identifica funções puras (geralmente indicando que não geram efeitos colaterais e sempre retornam o mesmo valor para os mesmos argumentos). É tratada especificamente por *PURE_FUNCTION_Decl*.
- **PROCEDURE**: Declara um procedimento. Diferente das funções, procedimentos não possuem tipo de retorno (*ARROW Type*), servindo apenas para executar uma sequência de comandos (*Scope*). É tratado por *PROCEDURE_Decl*.

A regra *Main* trata especificamente da função principal do programa. Ela mapeia uma assinatura do tipo *FUNCTION MAIN (...)* diretamente para a sintaxe padrão do C (*int main()*), concatenando essa

assinatura ao código gerado dentro do seu escopo (\$8->code) e gerando o registro final do programa executável.

2.5 Verificações realizadas

Para a análise semântica são realizadas verificação de escopo e declaração de variáveis, bem como verificação de compatibilidade de tipos, validação de arrays e tipos complexos, além da geração de código e desvios seguros.

Dessa forma, para a verificação de escopo e declaração de variáveis o compilador gerencia escopos utilizando as funções PushScope e PopScope acionadas pela regra Scope, garantindo que variáveis declaradas dentro de um bloco { ... } não vazem para o escopo externo. De outra forma, na regra IDs (usada em expressões), a função checkVarScope(\$1) garante que nenhuma variável seja utilizada sem antes ter sido declarada.

Em seguida, para a verificação e compatibilidade de tipos na regra Statements, o compilador valida se múltiplos comandos de retorno (Return) dentro do mesmo bloco possuem tipos compatíveis através da função checkTypeCompat(..., BOTH), de modo que, se houver tipos conflitantes, o compilador interrompe a execução com um erro fatal ("Returns with incompatible types;"). Vale salientar ainda que funções como handleOperandTypes são injetadas em todas as regras de expressões aritméticas e lógicas (OR, AND, NOT_EQUAL, comparações, somas, etc.), permitindo a garantia que os operadores sejam aplicados a operandos válidos e definem o tipo resultante da expressão (como forçar o tipo bool_ após uma comparação). Por fim, a regra SubprogramCall utiliza a função checkParamType(\$1, \$2->paramsTypes). Ela valida se os argumentos passados na chamada da função correspondem em ordem e tipo aos parâmetros esperados pela assinatura da função previamente guardada.

Isso posto, em relação à validação de arrays e tipos complexos, o compilador faz um loop para validar o acesso a índices (ArrayAccesses). Ele busca na tabela de tipos (lookup_symbol na typeTable) para checar se a variável sendo indexada é de fato um array (isArray) e para determinar o tipo final do dado extraído. Além disso, na regra de tipos [Type ; VALUE_INT], o compilador gera uma assinatura textual única (ex: [int;5]) e verifica se esse tipo de array já foi registrado. Caso não tenha sido, ele insere dinamicamente na tabela de tipos com a função allocTypeArray.

Por fim, para a geração de código e desvios seguros, embora não seja uma validação puramente estática, o sistema de controle de fluxo de estruturas como IF, ELSE e WHILE gera rótulos numéricos únicos (ifCount(), whileCount()). Isso mitiga erros semânticos na geração do código intermediário (C simplificado), impedindo que desvios (goto) saltem acidentalmente para blocos errados. .

3. INSTRUÇÕES DE USO DO COMPILADOR

Abaixo seguem as instruções de uso do compilador, para executar e gerar os binários faça:

```
# 1. Clone o repositório
git clone https://github.com/BlackbirdBlina/Engenharia-de-Linguagem.git
```

```
cd EngenhariaLinguagem

# 2. Instale o Fast Lexical Analyzer Generator (Flex)
sudo apt-get install flex

# 3. Use o makefile para gerar os binários
make

# 4. Execute os binários gerados
make run
```

Após gerar os binários seguindo os passos anteriores, basta executar o comando abaixo para compilar:

```
make testc
```